

Plan: Concentric-Pyramid Multi-Layer (CPML) for Signed-Graph Link Prediction and Cross-Query Hypergraph Convolution (XHC) for HyMeYOLO

2026-05-11

Scope

Two architectural proposals, sharing a hypergraph-convolution lineage:

- **CPML** — a vertex-degree-stratified depth network for signed-graph link prediction, targeting the Epinions ceiling (current 0.815, SGT ~ 0.95 , gap -0.13).
- **XHC** — cross-query hypergraph convolution for HyMeYOLO, replacing the current independent-query head with one that propagates information between overlapping query polygons.

Both are motivated by your reformulation of CPG: “going to the center, more cycles are detected, leading to abstraction.” That sentence describes a *multi-layer pyramid*, not a per-vertex cap allocation (which is what the earlier CPG implementation did and which we archived). The biological analogue is the cortical hierarchy $V1 \rightarrow V2 \rightarrow V4 \rightarrow IT$ — each level integrates over a larger receptive field. CPML applies that hierarchy to signed-graph cycles.

1. CPML: Concentric-Pyramid Multi-Layer

1.1 Definitions

Let $G = (V, E, \sigma)$ be a signed graph with vertex set V , edge set E , and edge-sign function $\sigma : E \rightarrow \{-1, +1\}$. Let $d : V \rightarrow \mathbb{N}$ be the (undirected) degree function.

Tier stratification. Choose tier breakpoints $0 = p_0 < p_1 < \dots < p_L = 1$ in degree-percentile space. Tier T_ℓ consists of all vertices whose degree percentile lies in $(p_{\ell-1}, p_\ell]$:

$$T_\ell = \{v \in V : F_d^{-1}(p_{\ell-1}) < d(v) \leq F_d^{-1}(p_\ell)\}.$$

Convention: T_L is the *innermost* (highest-degree) tier — “the centre”. T_1 is the outermost (lowest-degree) — “the periphery”.

Tier-restricted cycle pool. Let $\mathcal{C}_k(G)$ be the set of all simple closed k -cycles in G . Define the ℓ -tier cycle pool as

$$\mathcal{C}_\ell = \bigcup_{k \in \{3,4,5\}} \{c \in \mathcal{C}_k(G) : c \cap T_\ell \neq \emptyset\},$$

i.e. all cycles that touch at least one vertex of T_ℓ . Note: inner tiers’ cycle pools *strictly contain* more cycles than outer ones in expectation (hubs are incident to more cycles than leaves), giving the “more cycles toward the centre” property.

Tier-restricted incidence. Let $M_\ell \in \{-1, 0, +1\}^{|V| \times |\mathcal{C}_\ell|}$ be the signed-incidence matrix: $M_\ell[v, c] = 0$ if $v \notin c$, otherwise the product of signs along c from v ’s perspective.

1.2 Forward equations

CPML stacks L HSiKAN layers. Layer ℓ operates only on the ℓ -tier cycle pool:

$$H_\ell = \text{HSiKAN}_\ell(M_\ell, X_\ell), \quad X_\ell = \text{concat}(X_0, H_1, \dots, H_{\ell-1}),$$

where $X_0 \in \mathbb{R}^{|V| \times d_0}$ is the initial vertex feature and each X_ℓ concatenates the original features with all *outer-tier* embeddings produced so far. This is the “inward funnelling” — abstraction at the centre receives all peripheral signal.

Edge predictor. For a candidate edge (u, v) ,

$$s(u, v) = \text{MLP}(\text{concat}(H_L[u], H_L[v])),$$

where H_L is the final (innermost-tier) embedding. The model classifies the edge sign via $\text{sigmoid}(s(u, v))$.

1.3 Mathematical properties

Receptive field. Each HSiKAN layer aggregates over cycles touching its tier’s vertices. A leaf vertex in T_1 is in $\mathcal{C}_1$ only; a hub vertex in T_L participates in every tier’s cycles. So inner-tier embeddings have *strictly larger* effective receptive field — this mirrors the cortical $|RF|$ growth (Hubel–Wiesel: V1 a few degrees, V2 several, V4 large, IT scene-level).

Computational cost. Let $n_\ell = |T_\ell|$ and $m_\ell = |\mathcal{C}_\ell|$. CPML’s forward cost is $\sum_\ell O(n_\ell \cdot d_{\text{hidden}} \cdot m_\ell)$, compared to the flat HSiKAN’s $O(|V| \cdot d_{\text{hidden}} \cdot |\mathcal{C}|)$. For a power-law degree distribution (typical of social graphs), inner tiers have small n_ℓ but large m_ℓ ; outer tiers have large n_ℓ but small m_ℓ . The total cost is roughly the same order as flat HSiKAN; the speedup or slowdown depends on the percentile cuts.

Information bottleneck. The concat-style update $X_\ell = \text{concat}(X_0, H_1, \dots, H_{\ell-1})$ ensures information from every outer tier reaches every inner tier. There is no information loss in the forward direction. Inner tiers may overfit if n_L is small; mitigated by L_2 regularisation on the inner layers.

Connection to CNN-FPN. Replace “vertices” with “pixels”, “degree” with “feature-magnitude / saliency”, “cycles” with “receptive fields”, and CPML becomes an FPN-style multi-scale aggregator. The mathematical structure is identical: hierarchical aggregation over a stratified vertex/pixel set with inward feature propagation.

Biological analogue. $V1 \rightarrow V2 \rightarrow V4 \rightarrow \text{IT}$ forms a 4-stage hierarchy with receptive-field sizes $1 \rightarrow 5 \rightarrow 30 \rightarrow 60$. CPML with $L = 4$ mirrors this. Alternative: hippocampal CA3 attractor has a similar “periphery $\rightarrow$ centre $\rightarrow$ pattern completion” flow.

1.4 Hyperparameters

- $L \in \{2, 3, 4, 5\}$: number of tiers. Default $L = 3$.
- Percentile cuts: default $\{0, 0.2, 0.8, 1.0\}$ for $L = 3$ (bottom 20% leaves, middle 60% mid, top 20% hubs).
- Per-tier d_{hidden} : default constant. Could grow inward as $d_\ell = 2^{\ell-1} \cdot d_0$.
- Cycle caps per tier: $m_\ell \in [16, 256]$, possibly growing inward.

2. XHC: Cross-Query Hypergraph Convolution (HyMeYOLO)

2.1 Hypergraph formulation

For an image with N queries, each with K_q corners:

- Vertex set $V_{\text{img}} = \bigcup_{q=1}^N \{c_{q,1}, \dots, c_{q,K_q}\}$ (concatenated corners across queries).
- Hyperedge set $E_{\text{img}} = \{e_1, \dots, e_N\}$ where e_q contains $\{c_{q,1}, \dots, c_{q,K_q}\}$.
- Incidence $H_{\text{img}} \in \{0, 1\}^{|V_{\text{img}}| \times N}$ with $H_{\text{img}}[v, q] = 1$ iff corner v is in query q .

2.2 Forward equation (Feng et al. 2019 HGNN)

$$X^{(l+1)} = \sigma(D_v^{-1/2} H_{\text{img}} W D_e^{-1} H_{\text{img}}^\top D_v^{-1/2} X^{(l)} \Theta^{(l)}),$$

where D_v and D_e are the vertex / hyperedge degree diagonal matrices; W is a learnable diagonal hyperedge weight (one per query); $\Theta^{(l)}$ is the per-layer linear projection; $\sigma = \text{KAN-style univariate activation}$.

This propagates: corner $\rightarrow$ query (aggregate over corners) $\rightarrow$ corner (re-distribute). Overlapping corner positions across queries trigger lateral information exchange.

2.3 Subtlety: corners are continuous, not discrete

In the formulation above, vertices are *corner indices*, not *positions*. Two queries can predict corners at the same position without sharing a vertex. To get true cross-query propagation we need a *position-aware* hypergraph:

Soft incidence. Replace $H_{\text{img}}[v, q] \in \{0, 1\}$ with a position-similarity kernel:

$$H_{\text{img}}[v, q] = \kappa(c_v, c_q), \quad \kappa(p, q\text{-centre}) = \exp\left(-\frac{\|p - q\text{-centre}\|^2}{2\sigma_q^2}\right),$$

where σ_q is a learnable per-query scale. Now corners near a query’s centre receive a strong soft incidence even if they were predicted for a different query.

2.4 Computational cost

For N queries with K corners each, $|V_{\text{img}}| = NK$, $|E_{\text{img}}| = N$. The full HGNN forward is $O(NK \cdot d^2 + N^2 K \cdot d)$ per layer. For our HyMeYOLO scale ($N = 6\text{--}8$, $K = 4\text{--}8$, $d = 32$), this is $\sim 10^4$ flops per image — negligible compared to backbone.

Affected files and CORE.YAML compliance

CPML:

- NEW: `signedkan.wip/src/cpml.py` — model + tier stratification utility.
- NEW: `signedkan.wip/tests/test_cpml.py` — unit + smoke.
- MODIFY: `signedkan.wip/src/run_final_cell.py` — add `--model CPML` dispatch.

XHC:

- NEW: `signedkan.wip/src/vision/hymeyolo_xhc.py`.
- NEW: `signedkan.wip/tests/test_xhc.py`.
- MODIFY: `signedkan.wip/src/vision/train_circles_ricci.py` — add `(+xhc)` as 7th config.

CORE.YAML items touched: none. Both are additive new modules.

Test plan (CLAUDE.md §3)

CPML unit tests (`tests/test_cpml.py`):

- `tier_stratify_returns_correct_sizes` — on a power-law graph, verify $|T_\ell|$ ratios match percentile cuts.
- `forward_shapes_l2_l3_l4` — each $L \in \{2, 3, 4\}$.
- `backward_no_nan` — gradients flow.
- `l1_reduces_to_flat_hsiKAN` — $L = 1$ recovers the flat baseline numerically.
- `innermost_tier_sees_all_outer_features` — assert X_L row contains contributions from $X_0, H_1, \dots, H_{L-1}$.

CPML smoke (single seed, Bitcoin OTC, 10 epochs): expect $L = 2$ matches flat HSiKAN within ± 1 pp AUC (sanity); $L = 3$ to $L = 4$ shows measurable difference.

CPML real-data validation : queue 5-seed paired on Bitcoin OTC + Epinions (CPML-L3 vs flat HSiKAN-kitchen-sink) only *if* smoke shows $L \geq 3$ produces nontrivial differences.

XHC unit tests :

- `soft_incidence_normalises_to_one` — per-query.
- `forward_shapes_match_existing_head` — drop-in compat.
- `backward_no_nan`.
- `10_soft_incidence_reduces_to_per_query_pool` — Gaussian-kernel with $\sigma \rightarrow 0$ recovers the existing independent-query baseline.

Performance budget

- CPML peak RSS: bounded by flat HSiKAN $\times L$ (worst case); in practice expect $\leq 1.5\times$ due to tier-restricted pools.
- CPML wall time at $L = 3$: $\leq 1.8\times$ flat HSiKAN.
- XHC wall time: $\leq 1.1\times$ existing HyMeYOLO head (HGNN layer is cheap at $N = 6\text{--}8$ queries).

Risk anticipation

- **CPML inner-tier overfit**: $|T_L|$ may be a few hundred vertices on Bitcoin OTC. Mitigation: L_2 on H_L .
- **CPML cycle-pool overlap**: $\mathcal{C}_\ell$ may have huge overlap with $\mathcal{C}_{\ell-1}$, weakening the stratification. Verify via $|\mathcal{C}_\ell \setminus \mathcal{C}_{\ell-1}|$ in unit test.
- **XHC over-smoothing**: standard HGNN warning. Mitigate with 1 or 2 layers max.
- **XHC soft incidence numerical instability**: learnable σ_q can drift to 0, making the kernel degenerate. Constrain via $\sigma_q = \text{softplus}(\hat{\sigma}_q)$.

Rollback path

Both modules are additive new files. Disabling means not invoking them from the training script. No core math kernels touched; no existing API breaks.

Feasibility-gate criteria

Before queueing a real training run for either architecture:

1. Unit tests all pass (`cargo test` N/A here; `pytest signedkan_wip/tests/test_{cpml,xhc}.py`).
2. Smoke 1-epoch run produces a non-NaN loss curve.
3. Memory peak ≤ 4 GB at default hyperparams.
4. Forward + backward shapes match expectations.

If any criterion fails, halt and re-design.

Sequencing

1. Land CPML module + unit tests + smoke (this session).
2. Wait for current Stage 7 + 7.5 + 9 + 8 queue to complete.
3. If smoke green, queue CPML-L3 1-seed on Bitcoin OTC + Epinions as a Stage 10.
4. XHC follows the same gate-then-queue protocol; lower priority because Slashdot SOTA already broken on the vision-flavoured side of the broader work.